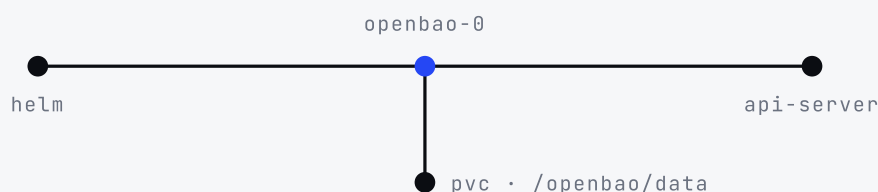


OPENBAO ON KUBERNETES

A secrets store inside the cluster: Helm, Raft, init and unseal, Kubernetes auth – and what is still missing afterwards.



OpenBao on Kubernetes

A secrets store inside the cluster: Helm, Raft, init and unseal, Kubernetes auth – and what is still missing afterwards

Thomas Zachmann

17 September 2026

Contents

What this is about

- Why by hand, and why without AI
- A word about the values in this guide
- License and liability
- Who this is for
- The building blocks
- The architecture in one picture

Part I – Helm

- The chart
- The values
- Installing
- What helm upgrade does *not* do later

Part II – init and unseal

- Why this is not automated
- init
- unseal
- What happens after every restart
- The first login

Part III – Kubernetes auth

- How it works
- Enable and configure
- A role for one consumer
- Checking without ESO

Part IV – Access from outside

- The path
- The bao CLI from outside
- The UI

Part V – The current state, honestly

Part VI – Hardening: what comes next

1. Remove the token file from the pod
2. Audit device
3. Admin policy and userpass
4. Prove the replacement access – do not skip
5. Revoke the root token
6. Snapshots
7. Configuration into OpenTofu

Part VII – What can go wrong

- Pod stays 0/1 Running

Log full of service_registration ... 403

helm upgrade changes nothing

403 permission denied on everything

Kubernetes login fails

OpenBao stops accepting requests, log mentions audit

After node maintenance: sealed

Appendix A – All commands

Appendix B – Glossary

What this is about

A Kubernetes cluster needs a place for secrets that can do more than kind: `Secret` – a place that logs access, issues credentials with an expiry, and tells workloads apart by their identity. This guide builds exactly that: **OpenBao inside the cluster**, as a `StatefulSet` with Raft storage, installed via Helm, initialised and unsealed by hand, with Kubernetes auth as the login method for workloads.

It is the first part of a series. The second, *Dynamic Database Credentials with OpenBao*, builds on precisely this setup and lets an application draw short-lived PostgreSQL access from it. The third describes the same OpenBao on a virtual machine – with Ansible, OpenTofu and a rehearsed disaster recovery. Many decisions in this guide are taken from there, and wherever the cluster setup falls *behind* the VM setup, that is said explicitly.

Everything here was actually carried out on an RKE2 cluster in a homelab. The state at the end is described honestly – including what has not been done yet.

Why by hand, and why without AI

In a real setup a Helm release is not deployed by hand but via GitOps, and the OpenBao configuration not via CLI but via OpenTofu. Even so, this guide walks every step once by hand – and deliberately without AI assistance as a shortcut. Whoever lets a language model write the values gets a running OpenBao and no idea why `tlsDisable` is set, what distinguishes Raft from `file`, or why the pod is sealed after every restart.

The yardstick: whoever has worked through this guide can draw, on a blank sheet, how a pod in the cluster gets to a secret in OpenBao – through which objects, with which token, verified by whom. Whoever can do that can also hook the Helm release into Flux or Argo and move the configuration to OpenTofu.

Tools like [nyrvex](#), which generates the configuration of an AI platform's secret store and identity provider, take these steps off your hands later. You should have walked them yourself once, to be able to judge what was generated.

A word about the values in this guide

The cluster is a development environment: no access from outside, no real data. Hostnames and internal addresses are replaced by placeholders (`bao.example.internal`). Unseal keys and the root token appear nowhere – they are not in the original terminal log either, because `init` deliberately never runs inside an automation or a chat window.

License and liability

This guide is licensed under CC BY 4.0: it may be copied, shared and adapted, commercially too, as long as the author is credited. It is provided as is, without warranty. Everything in it was carried out in a development environment; whoever reproduces it elsewhere does so at their own risk.

Who this is for

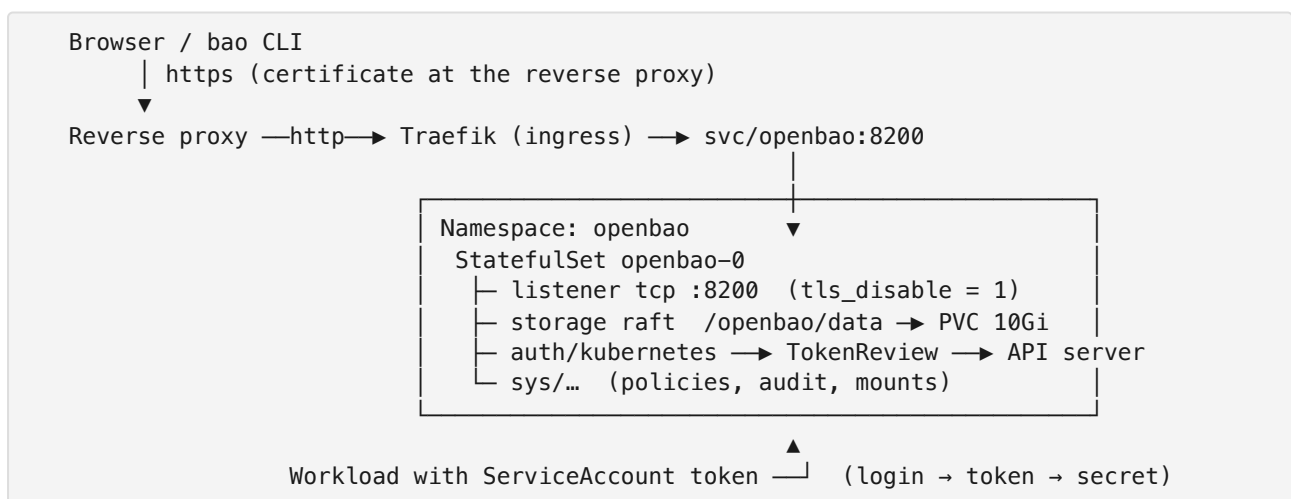
Readers who know Kubernetes basics (StatefulSet, PVC, Ingress, ServiceAccount, Helm) and know what a secrets manager is. OpenBao is a fork of HashiCorp Vault; everything here applies to both, only the CLI name (bao) and the environment variables (BAO_ADDR) differ.

If you want to understand the concepts – seal/unseal, auth methods, policies, secrets engines, leases – from the ground up, with labs that run on a laptop, that is in my book **Vault in Practice** (Leanpub, leanpub.com/vault-in-practice). This guide assumes those concepts and shows how they come together in a cluster.

The building blocks

Component	Version	Role
RKE2 Kubernetes	v1.36.4+rke2r1	3 control-plane nodes, 3 workers, Traefik as ingress, Longhorn as storage
OpenBao Helm chart openbao/openbao	chart 0.29.4, app 2.6.2	The deployment
Helm	3.16	Installs the chart
bao CLI	2.6.2	Included in the pod; local install optional

The architecture in one picture



Three things to take from the picture:

1. **TLS ends before the cluster.** The listener speaks plain HTTP. That is a deliberate simplification for a homelab behind a reverse proxy – and the first thing to change in a production environment.

2. **State lives in a PVC.** Raft writes to /openbao/data. Lose the volume and you lose everything – which is why snapshots are not a convenience but a duty.
3. **Workloads log in with what they already have:** their ServiceAccount token. OpenBao asks the API server whether the token is genuine. No second secret has to be distributed.

Part I – Helm

The chart

OpenBao maintains its own chart, derived from the Vault chart. It knows three modes: dev (in-memory, unsealed – for playing only), standalone (one pod, persistent volume) and ha (several pods with Raft quorum). For a homelab on a single physical host, standalone is the right choice – three pods on the same host do not protect against the dominant failure (host down) and triple the effort of manual unsealing.

```
helm repo add openbao https://openbao.github.io/openbao-helm
helm repo update
helm search repo openbao/openbao --versions | head -3
```

The values

The full file (k8s/values.yaml) is 70 lines; here are the parts with their reasoning.

```
global:
  tlsDisable: true
```

TLS is terminated at the reverse proxy. The chart uses this switch in several places: it sets BAO_ADDR inside the pod to http://, configures the probes without TLS and omits the tls block from the ingress. Setting only the listener to tls_disable = 1 but not global.tlsDisable leaves the readiness probes pointing nowhere.

```
injector:
  enabled: false
```

The sidecar injector (vault-k8s) writes secrets as files into pods. For the planned route – the External Secrets Operator – it is not needed. Every controller that does not run is one fewer holding permissions in the cluster.

```
server:
  standalone:
    enabled: true
    config: |
      ui = true

      listener "tcp" {
        address      = "[::]:8200"
        cluster_address = "[::]:8201"
        tls_disable   = 1
      }

      storage "raft" {
        path = "/openbao/data"
      }
```

The most important intervention is storage "raft". The chart default for standalone is storage "file" – a backend that **cannot take snapshots**. bao operator raft snapshot save only exists with Raft. Whoever keeps the default has no way to take a consistent backup other than copying the PVC – and that is not consistent while the process is running.

api_addr and cluster_addr are deliberately absent: the chart sets BAO_API_ADDR and BAO_CLUSTER_ADDR as environment variables on the StatefulSet, also in standalone mode, and OpenBao reads them at startup.

service_registration "kubernetes" was in this block in the original setup – and is deliberately **no longer** part of k8s/values.yaml. The registration is meant to write the state (active, sealed, ...) as labels onto the pod so that an HA service selects only the active node. In standalone mode, however, the chart renders neither the Role nor the RoleBinding for that, and OpenBao then logs every five seconds:

```
[WARN] service_registration.kubernetes: unable to set initial state due; will retry:
err="GET https://10.43.0.1:443/api/v1/namespaces/openbao/pods/openbao-0 ...
resp statusCode: 403"
```

For 22 hours before anyone noticed. In standalone mode the services select on component: server with publishNotReadyAddresses: true anyway – the labels would change nothing. So leave it out, or add a Role with get/patch on pods if you want the labels for monitoring.

```
dataStorage:
  enabled: true
  size: 10Gi
  storageClass: longhorn

persistentVolumeClaimRetentionPolicy:
  whenDeleted: Retain
  whenScaled: Retain
```

Both explicit, although they would be the defaults. The reason is the same as for ingressClassName: a changed cluster default must not silently move the Raft data elsewhere or take it along on helm uninstall.

```
ingress:
  enabled: true
  ingressClassName: traefik
  hosts:
    - host: bao.example.internal
      paths: []

ui:
  enabled: true
  serviceType: ClusterIP
```

No tls block, no cert-manager. The reverse proxy in front of the cluster holds the certificate and forwards to Traefik over HTTP.

Installing

```
helm upgrade --install openbao openbao/openbao \
  --version 0.29.4 \
  --namespace openbao --create-namespace \
  --values k8s/values.yaml
```

Without `--wait`. The chart's readiness probe is `bao status`, which fails as long as OpenBao is sealed. On a fresh installation the pod is only *Ready* after `init + unseal` – `--wait` would just run into its timeout.

Check beforehand without changing anything:

```
helm upgrade --install openbao openbao/openbao --version 0.29.4 \
  --namespace openbao --values k8s/values.yaml --dry-run=server
```

Afterwards:

```
$ kubectl get sts,svc,ingress,pvc -n openbao
```

NAME	READY	AGE
statefulset.apps/openbao	0/1	1m

NAME	TYPE	CLUSTER-IP	PORT(S)
service/openbao	ClusterIP	10.43.179.240	8200/TCP,8201/TCP
service/openbao-internal	ClusterIP	None	8200/TCP,8201/TCP
service/openbao-ui	ClusterIP	10.43.158.186	8200/TCP

NAME	CLASS	HOSTS	PORTS
ingress.networking.k8s.io/openbao	traefik	bao.example.internal	80

NAME	STATUS	CAPACITY	STORAGECLASS
persistentvolumeclaim/data-openbao-0	Bound	10Gi	longhorn

READY 0/1 is correct here. The pod runs, OpenBao answers, but it is not initialised – and the chart rightly calls that “not ready”.

What `helm upgrade` does *not* do later

The chart sets `updateStrategyType: OnDelete`. A changed `values.yaml` becomes a new `ConfigMap`, but the pod is **not** restarted. Only `kubectl -n openbao delete pod openbao-0` applies the change – and afterwards OpenBao is sealed. It is the same trade-off as a configuration reload on a VM: a restart costs an unseal, so it only happens on purpose.

Part II – init and unseal

Why this is not automated

`bao operator init` generates the unseal keys and the root token. That output appears **exactly once**. If `init` runs in a pipeline, in Ansible or in an AI chat, the key material ends up in logs, in task output or in a context window. Hence: a normal terminal, a shell inside the pod, output straight into the password manager, then close the terminal.

init

```
kubectl exec -it -n openbao openbao-0 -- sh
/ $ bao operator init -key-shares=5 -key-threshold=3
```

Shamir 5 of 3: five keys, three suffice to unseal. In a homelab where one person holds all five, that is no security gain over 1/1 – but it is the format a later move to several custodians expects, and it costs nothing.

The output contains `Unseal Key 1` to `5` and `Initial Root Token`. All of it into the password manager. **Not** into this repo, **not** onto the NAS where the snapshots will later live – the snapshots are encrypted with exactly these keys, and whoever keeps both in one place has lost both when that place is gone.

unseal

```
/ $ bao operator unseal      # enter key, no echo
/ $ bao operator unseal
/ $ bao operator unseal
/ $ bao status
```

Key	Value
---	-----
Seal Type	shamir
Initialized	true
Sealed	false
Total Shares	5
Threshold	3
Version	2.6.2
Storage Type	raft
HA Enabled	true
HA Cluster	https://openbao-0.openbao-internal:8201
HA Mode	active

`HA Enabled true` on a single pod is surprising – it is a property of Raft, not of the operating mode. The pod is the active (and only) node.

Without an argument, `bao operator unseal` prompts for the key interactively and reads it without echo. A key passed as an argument would sit in the shell history and be visible to other processes via `/proc`.

After the unseal the pod switches to READY 1/1.

What happens after every restart

The pod dies, gets rescheduled, the node is patched: OpenBao starts **sealed**. That is the deliberately accepted price of the Shamir decision. As long as nothing depends on it, it is merely inconvenient. Once the External Secrets Operator hangs off it (N° 2), every restart is an outage for everything that has to renew secrets. At that point auto-unseal (transit engine on the VM OpenBao, or a cloud KMS) belongs on the list.

The first login

```
/ $ bao login  
Token (will be hidden):
```

The root token. From here the shell is root in OpenBao – and `bao login` writes the token to `~/.vault-token` inside the container. More on that in Part V; for now, this is the state in which the configuration is done.

Part III – Kubernetes auth

How it works

A pod has a ServiceAccount token (projected, with an expiry). The pod sends it to `auth/kubernetes/login`. OpenBao forwards it to the API server (TokenReview), gets back namespace and name of the ServiceAccount, compares them with the role, and issues an OpenBao token with the role's policies.

The trick: no secret has to be distributed. The pod has its token anyway, and OpenBao has its own pod token and the cluster CA to make the TokenReview.

Enable and configure

```
/ $ bao auth enable kubernetes
/ $ bao write auth/kubernetes/config \
    kubernetes_host="https://kubernetes.default.svc:443"
```

Nothing more. Because OpenBao itself runs in the cluster, it uses its own ServiceAccount token and the CA from `/var/run/secrets/kubernetes.io/serviceaccount/` for the TokenReview. The chart provides the permission for that: it renders a ClusterRoleBinding of the ServiceAccount `openbao` to the built-in ClusterRole `system:auth-delegator` – exactly the right to submit TokenReviews, and nothing else. If OpenBao runs **outside** the cluster, `kubernetes_ca_cert` and `token_reviewer_jwt` must be set explicitly – and the reviewer token needs that same ClusterRole.

A role for one consumer

A role binds a ServiceAccount to policies:

```
/ $ bao write auth/kubernetes/role/eso-demo \
    bound_service_account_names=demo \
    bound_service_account_namespaces=demo \
    token_policies=eso-demo \
    token_ttl=1h
```

```
/ $ bao read auth/kubernetes/role/eso-demo
Key                               Value
---                               -
alias_name_source                  serviceaccount_uid
bound_service_account_names        [demo]
bound_service_account_namespaces   [demo]
token_policies                     [eso-demo]
token_ttl                          1h
```

alias_name_source: serviceaccount_uid means: the identity in OpenBao hangs on the UID of the ServiceAccount. If the ServiceAccount is deleted and recreated, it is a new identity from OpenBao's point of view – the old alias remains as a corpse in the identity store until someone removes it.

What the policy eso-demo allows is the consumer's business – in N° 2 those are KV paths and database/creds/*. For this guide it suffices: the role says *who* may log in and *which* policy they get.

Checking without ESO

The quickest test is a pod with the ServiceAccount that performs the login by hand:

```
kubectl create namespace demo
kubectl create serviceaccount demo -n demo
kubectl run -n demo --rm -it --restart=Never \
  --overrides='{ "spec": { "serviceAccountName": "demo" } }' \
  --image=curlimages/curl:8.11.1 probe -- sh
```

Inside the pod:

```
JWT=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
curl -s http://openbao.openbao.svc:8200/v1/auth/kubernetes/login \
  -d '{"role":"eso-demo","jwt":"$JWT"}' | head -c 400
```

A client_token in the response means: auth method, TokenReview and role work. A 403 permission denied almost always means: namespace or name of the ServiceAccount do not match the role – not that the policy forbids something, because the policy only comes into play after the login.

Part IV – Access from outside

The path

```
Browser —https→ reverse proxy (certificate) —http→ worker node:80 (Traefik hostPort)
                                     |
                                     v
                                Ingress openbao → svc/openbao:8200
```

In RKE2, Traefik runs as a DaemonSet with hostPort 80/443 – but only on the workers, because the control-plane nodes are tainted and the DaemonSet does not tolerate the taints. The reverse proxy's target is therefore the IP of one worker. If that worker goes down, the UI is unreachable although OpenBao keeps running.

A floating VIP (MetalLB) would solve that. It was deliberately not installed: the availability of the *UI* is not worth another controller in a homelab – the workloads in the cluster reach OpenBao via `svc/openbao`, and that does not depend on any particular worker.

The bao CLI from outside

```
export BAO_ADDR=https://bao.example.internal
bao status
bao login    # token
```

Two traps that cost time while debugging:

- A terminal with `BAO_ADDR` pointing at the ingress but no token: **every** call answers 403 permission denied – even `bao token capabilities`. It looks like a policy problem and is a missing login. `bao token lookup` first.
- `bao login` writes to `~/.vault-token` – the same path the Vault CLI uses. Whoever runs a Vault or a second OpenBao in parallel overwrites one system's token when logging in to the other. A puzzling 403 is then usually a token from the other system.

The UI

`https://bao.example.internal/ui/` – login with the root token or, after hardening, by username. The UI is handy for an overview of mounts, policies and leases; for anything reproducible the CLI is better, because its calls can move into a script or into OpenTofu.

Part V – The current state, honestly

This is where this guide falls behind N° 3. On the VM the bootstrap sequence has been carried out in full and verified. In the cluster it has **not**. This is what it looks like:

```
/ $ bao audit list
No audit devices are enabled.

/ $ bao auth list
Path          Type          Description
----          -
kubernetes/   kubernetes    n/a
token/        token         token based credentials

/ $ bao policy list
default
eso-demo
root

/ $ bao token lookup | grep -E 'display_name|policies'
display_name    root
policies        [root]

/ $ ls -la ~/.vault-token
-rw----- 1 openbao openbao /home/openbao/.vault-token
```

Four findings:

- 1. No audit device.** Nobody can answer who read which path when. For a secrets store, that is the main reason to run one at all.
- 2. The root token is the only administrative access.** There is no userpass, no admin policy. The root token never expires and cannot be scoped.
- 3. The root token sits as a file in the pod.** `bao login` wrote it to `/home/openbao/.vault-token`. Anyone with `kubectl exec` on the pod – that is, anyone with sufficient RBAC in the namespace `openbao` – is root in OpenBao. The file lives in the container filesystem and disappears on restart; until then it is there.
- 4. No snapshots.** The PVC is the only copy of the Raft data. Longhorn does replicate the volume across nodes, but that protects against a disk failure, not against an accidental `helm uninstall` with a deleted PVC, a broken upgrade or a slip with `bao delete`.

This is the state in which N° 2 was built. Acceptable for a demo, for nothing else. Part VI describes what to do – the commands are taken from the VM runbook (N° 3), where they have been rehearsed.

Part VI – Hardening: what comes next

The order is not arbitrary. Audit first, so that everything after is logged. Root token last, and only once the replacement access has been **proven**.

1. Remove the token file from the pod

Immediately, independent of the rest:

```
/ $ rm ~/.vault-token
```

For further work, hold the token only in the environment variable of the running shell:

```
/ $ read -rs BAO_TOKEN; export BAO_TOKEN
```

`read -rs` reads without echo and without a history entry. When the shell is closed, the token is gone.

2. Audit device

```
/ $ bao audit enable file file_path=/openbao/audit/audit.log
```

`/openbao/audit` is the path the chart provides as a separate PVC with `server.auditStorage.enabled: true` – without that volume the log sits in the container and is gone after a restart. So first extend the values:

```
server:
  auditStorage:
    enabled: true
    size: 2Gi
    storageClass: longhorn
```

Then `helm upgrade`, delete the pod, unseal, then `audit enable`.

Two things to know: if OpenBao cannot write to the audit log, it **refuses requests**. A full volume takes the secrets store off the network. That is a security feature and, without rotation, a time bomb – on the VM `logrotate` handles it; in the cluster it takes a sidecar or a second audit device (`syslog`, socket to a log collector) that becomes the primary sink. And: from now on all entries show Traefik's IP instead of the real client, as long as `x_forwarded_for_authorized_addrs` is not set on the listener.

3. Admin policy and userpass

On the VM this comes from OpenTofu. In the cluster the fitting way is the same – a second Tofu workspace or a second directory, against `https://bao.example.internal`. Until that exists, the CLI version:

```

/ $ bao policy write admin - <<'EOF'
path "sys/health" { capabilities = ["read", "sudo"] }
path "sys/capabilities-self" { capabilities = ["update"] }
path "sys/mounts" { capabilities = ["read", "list"] }
path "sys/mounts/*" { capabilities = ["create", "read", "update", "delete",
    "list", "sudo"] }
path "sys/auth" { capabilities = ["read", "list"] }
path "sys/auth/*" { capabilities = ["create", "read", "update", "delete",
    "sudo"] }
path "sys/policies/acl" { capabilities = ["list"] }
path "sys/policies/acl/*" { capabilities = ["create", "read", "update", "delete",
    "list"] }
path "sys/audit" { capabilities = ["read", "list", "sudo"] }
path "sys/audit/*" { capabilities = ["create", "read", "update", "delete",
    "sudo"] }
path "sys/leases/*" { capabilities = ["create", "read", "update", "delete",
    "list", "sudo"] }
path "sys/storage/raft/snapshot" { capabilities = ["read"] }

# Break glass: OpenBao disables the unauthenticated sys/generate-root/*
# endpoints since 2.5.3. Without these paths this login cannot mint a
# replacement root token – and the unseal keys alone cannot either.
path "sys/generate-root-token/attempt" { capabilities = ["create", "read", "update", "delete",
    "sudo"] }
path "sys/generate-root-token/update" { capabilities = ["create", "update", "sudo"] }
path "sys/decode-token" { capabilities = ["create", "update"] }

path "auth/*" { capabilities = ["create", "read", "update", "delete",
    "list", "sudo"] }
path "identity/*" { capabilities = ["create", "read", "update", "delete",
    "list"] }
path "secret/*" { capabilities = ["create", "read", "update", "delete",
    "list"] }
path "database/*" { capabilities = ["create", "read", "update", "delete",
    "list"] }
EOF

/ $ bao auth enable userpass
/ $ bao write auth/userpass/users/admin \
    password="$(read -rs -p 'Password: ' p; echo "$p")" \
    token_policies=admin token_ttl=1h token_max_ttl=8h

```

The password belongs in the password manager **before** step 5. Under OpenBao it is recovery material, not a convenience: since 2.5.3 the unauthenticated `sys/generate-root/*` endpoints are off by default, and `bao operator generate-root` uses the authenticated `sys/generate-root-token` endpoints. Three unseal keys alone are therefore **no way back** – it takes an additional login with exactly these paths in its policy.

4. Prove the replacement access – do not skip

```
# -token-only: otherwise 'bao login' overwrites the running root token in
# ~/.vault-token BEFORE it is proven that the new access works.
/ $ ADMIN_TOKEN="$(bao login -method=userpass -token-only username=admin)"

/ $ BAO_TOKEN="$ADMIN_TOKEN" bao token lookup | grep policies
/ $ BAO_TOKEN="$ADMIN_TOKEN" bao policy list

# The break-glass path itself: -init starts a root generation, -cancel
# aborts it. That proves the capability without producing a root token
# you would then have to handle.
/ $ BAO_TOKEN="$ADMIN_TOKEN" bao operator generate-root -init
/ $ BAO_TOKEN="$ADMIN_TOKEN" bao operator generate-root -cancel

/ $ unset ADMIN_TOKEN
```

Only once admin appears in the policies, policy list works **and** the generate-root -init/-cancel pair goes through is the replacement access proven. If any of it fails: do **not** run step 5.

5. Revoke the root token

```
/ $ bao token lookup | grep policies      # [root]
/ $ bao token revoke -self
/ $ bao token lookup                      # must fail now
```

From here userpass is the way in. Break glass if that access is lost:

```
bao login -method=userpass username=admin
bao operator generate-root -init
bao operator generate-root      # 3x with unseal keys
```

6. Snapshots

On the VM: a systemd timer that runs bao operator raft snapshot save daily, verifies the archive and copies it to a NAS (N° 3 in detail). In the cluster the equivalent is a CronJob in the namespace openbao that, with its own ServiceAccount, a Kubernetes auth role and the policy

```
path "sys/storage/raft/snapshot" { capabilities = ["read"] }
```

takes the snapshot and writes it to object storage (S3-compatible, e.g. MinIO or the NAS). The advantage over the VM: no periodic token in a file – the CronJob logs in with its ServiceAccount on every run.

Until the CronJob exists, a snapshot by hand:

```
kubectl exec -n openbao openbao-0 -- \
  sh -c 'BAO_TOKEN=... bao operator raft snapshot save /tmp/bao.snap' && \
kubectl cp openbao/openbao-0:/tmp/bao.snap ./openbao-$(date +%Y%m%dT%H%M).snap
```

And the check OpenBao itself does not offer (raft snapshot inspect only exists in Vault): the archive is a gzipped tar with meta.json, state.bin and SHA256SUMS –

```
tar -tzf openbao-*.snap
tar -xz0f openbao-*.snap SHA256SUMS
tar -xz0f openbao-*.snap state.bin | sha256sum      # must match the line above
```

Rehearse a restore once before relying on it. An untested restore is an assumption, not a backup. The procedure – onto a fresh cluster, with –force and two different key sets in sequence – is in N° 3 and applies unchanged in the cluster.

7. Configuration into OpenTofu

Everything from Part III and this part is state that is lost on a rebuild. The objects and their resources:

Object	Tofu resource
Audit device	vault_audit
Policies admin, eso-demo	vault_policy
userpass + admin user	vault_auth_backend, vault_userpass_auth_backend_user (with password_wo, so the password never reaches the state)
Kubernetes auth	vault_auth_backend (type = "kubernetes"), vault_kubernetes_auth_backend_config, vault_kubernetes_auth_backend_role

The provider is called hashicorp/vault – there is no openbao/openbao provider. OpenBao kept the Vault HTTP API, the provider works unchanged and is pointed at OpenBao via VAULT_ADDR. What the directory looks like, including state encryption and handling of the admin password, is described in N° 3 and carries over one to one.

Part VII – What can go wrong

Pod stays 0/1 Running

Expected as long as OpenBao is sealed or not initialised. `kubectl exec ... bao status` shows which case it is. Only when `Sealed false` and the pod still does not become Ready is something broken – then look at the probe in the StatefulSet (`kubectl describe pod`).

Log full of `service_registration ... 403`

A warning every five seconds: `service_registration "kubernetes"` is configured, but the ServiceAccount may not patch its own pod. In standalone mode the chart does not render the required Role. Remove the block from the server config (see Part I) or create a Role with `get, update, patch` on pods and a RoleBinding to the ServiceAccount `openbao`.

`helm upgrade` changes nothing

`OnDelete`. Delete the pod, unseal. See Part I.

`403 permission denied on everything`

No token, expired token, or token from the wrong system. `bao token lookup` first. Only when that works and a specific path still returns 403 is it the policy – `bao token capabilities <path>` shows deny.

Kubernetes login fails

Message	Cause
<code>service account name not authorized</code>	<code>bound_service_account_names</code> does not match
<code>namespace not authorized</code>	<code>bound_service_account_namespaces</code> does not match
Post "https://kubernetes.default.svc:443/.../tokenreviews": ...	OpenBao cannot reach the API server, or its own token may not submit a TokenReview
<code>permission denied only when reading a path</code>	Login succeeded; the role's policy does not allow the path

OpenBao stops accepting requests, log mentions audit

The audit volume is full or not writable. OpenBao then refuses on purpose. Enlarge the volume or rotate the log – and afterwards set up a second audit device so that this does not happen again.

After node maintenance: sealed

Expected. `kubectl exec -it -n openbao openbao-0 -- bao operator unseal`, three times. If this happens too often: auto-unseal, see Part II.

Appendix A – All commands

```
# — Helm —————
helm repo add openbao https://openbao.github.io/openbao-helm
helm upgrade --install openbao openbao/openbao --version 0.29.4 \
  -n openbao --create-namespace -f k8s/values.yaml

# — init / unseal (shell in the pod, normal terminal) —————
kubectl exec -it -n openbao openbao-0 -- sh
bao operator init -key-shares=5 -key-threshold=3      # output -> password manager
bao operator unseal                                  # x3
bao status
read -rs BAO_TOKEN; export BAO_TOKEN                 # no 'bao login' in the pod

# — Kubernetes auth —————
bao auth enable kubernetes
bao write auth/kubernetes/config kubernetes_host="https://kubernetes.default.svc:443"
bao write auth/kubernetes/role/eso-demo \
  bound_service_account_names=demo bound_service_account_namespaces=demo \
  token_policies=eso-demo token_ttl=1h

# — Hardening (keep the order) —————
bao audit enable file file_path=/openbao/audit/audit.log
bao policy write admin - < admin.hcl
bao auth enable userpass
bao write auth/userpass/users/admin password=... token_policies=admin token_ttl=1h token_max
  _ttl=8h
ADMIN_TOKEN="$(bao login -method=userpass -token-only username=admin)"
BAO_TOKEN="$ADMIN_TOKEN" bao operator generate-root -init
BAO_TOKEN="$ADMIN_TOKEN" bao operator generate-root -cancel
bao token revoke -self                               # only after the test passed

# — Snapshot by hand —————
kubectl exec -n openbao openbao-0 -- sh -c 'bao operator raft snapshot save /tmp/bao.snap'
kubectl cp openbao/openbao-0:/tmp/bao.snap ./openbao-$(date +%Y%m%dT%H%M).snap
tar -xz0f openbao-*.snap state.bin | sha256sum
```

Appendix B – Glossary

Seal / Unseal – OpenBao starts with encrypted storage and without the key to it. Unseal reassembles the master key from Shamir shares; until then OpenBao only answers status.

Shamir 5/3 – The master key is split into five shares, any three reconstruct it.

Raft – Integrated storage backend with a consensus protocol. Sensible even with one node, because only Raft can take snapshots.

Root token – The token from `init`. Valid indefinitely, cannot be scoped. Revoked after the bootstrap and regenerated via `generate-root` when needed.

Auth method – How a client proves its identity. `kubernetes` verifies ServiceAccount tokens via `TokenReview`; `userpass` verifies passwords.

Role (auth) – Binds identity attributes (ServiceAccount, namespace) to policies and token properties.

Policy – Allowed paths and capabilities. Default: everything denied.

Audit device – Logs every request and response (with hashed secrets). If OpenBao cannot write, it refuses requests.

TokenReview – The Kubernetes API by which a third party has a ServiceAccount token checked for validity and ownership.

Break glass – The route to a new root token when the old one is gone. Under OpenBao it needs unseal keys **and** a login with `sys/generate-root-token/*`.

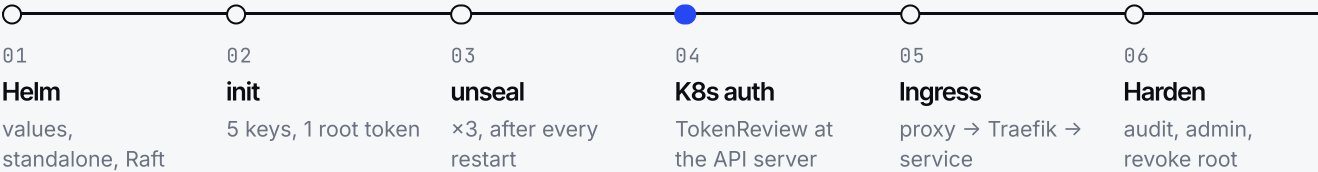
OnDelete – Update strategy of the StatefulSet: new configuration only takes effect when the pod is deleted by hand.

The pod is running. It is sealed. And that is exactly right.

This guide documents building an OpenBao inside a Kubernetes cluster as it actually happened: Helm values with their reasoning, init and unseal in a pod shell, Kubernetes auth with TokenReview, the path from outside through proxy and Traefik.

And the state afterwards, honestly: no audit device, the root token sitting as a file in the pod, a `service_registration` block that failed with 403 every five seconds for 22 hours. Plus the hardening that follows – in the order that rules out a lockout.

```
-- The one line that makes backups possible at all
storage "raft" {
  path = "/openbao/data"
}
```



kubernetes	RKE2 v1.36 · Traefik · Longhorn
openbao	2.6.2 · chart 0.29.4 · raft
helm	3.16 · standalone
auth	kubernetes · TokenReview
storage	10Gi PVC · Retain

25 pages · 7 parts · 2 appendices
Deep dive: **Vault in Practice**
leanpub.com/vault-in-practice
Set in Inter Tight, Inter and JetBrains Mono.
All output is real terminal output.
CC BY 4.0 · share freely · no warranty